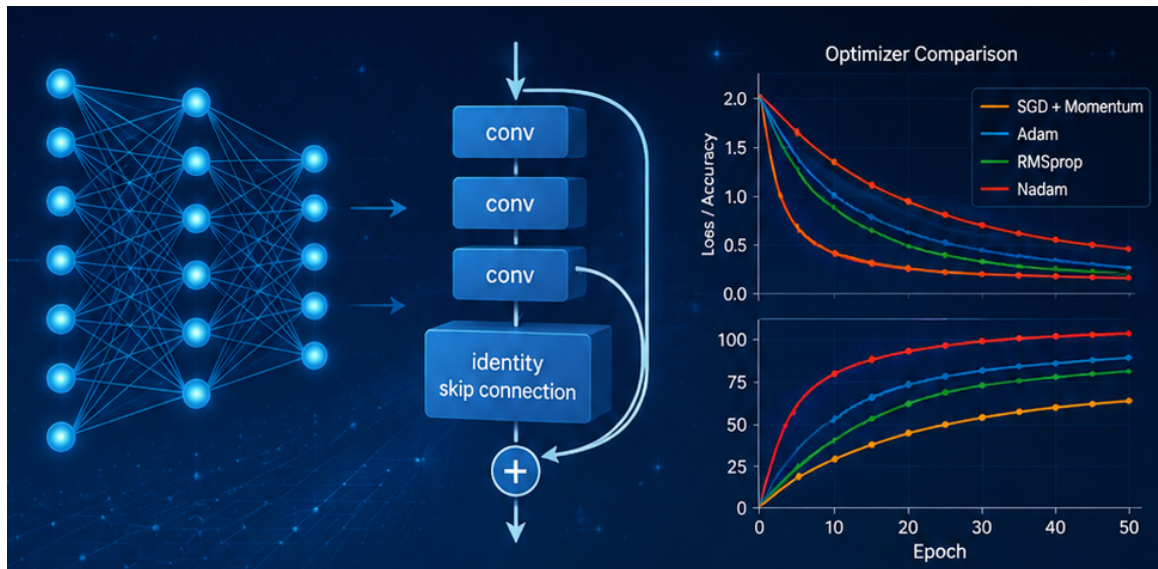


Master of Excellence in Artificial Intelligence

M1



END-OF-MODULE PROJECT REPORT:

Comparison of Optimizers for Training Deep Neural Networks

Case Study: ResNet-34 on CIFAR-100

Prepared by:

Miss. OUARRAK Layla

Miss. OUTIGHLI Sanae

Miss. RHAOUFAL Khadija

Supervised by:

Prof. Faouzia BENABBOU

Faculty of Sciences Ben M'Sick

Supervisor

Academic Year: 2025 – 2026

Contents

List of Tables	2
1 Introduction	3
1.1 Project Objective	3
1.2 Brief Presentation of Optimizers and Architectures	3
1.2.1 Overview of the Optimizers Studied	3
1.2.2 Model Architecture and Dataset	4
2 Literature Review	4
2.1 Classical Optimizers	4
2.1.1 Stochastic Gradient Descent (SGD)	5
2.1.2 SGD with Momentum	5
2.2 Adaptive Optimizers	5
2.2.1 Adagrad	6
2.2.2 RMSprop	6
2.2.3 Adam (Adaptive Moment Estimation)	6
2.2.4 Nadam (Nesterov-accelerated Adam)	7
2.3 Known Results on CIFAR-100	7
3 Experimental Methodology	8
3.1 Dataset: CIFAR-100	8
3.2 Model Architecture: ResNet-34	8
3.3 Tools and Framework	9
3.4 Experimental Protocol (Runs)	9
3.5 Measurement Protocol	10
4 Results	10
4.1 Learning Curves	10
4.2 Global Comparative Table	10
4.3 Analysis of Raw Results	11
5 Discussion	12
5.1 Convergence Speed and Stability	12
5.2 Generalization (Overfitting)	12
5.3 Computational Cost (Time/Memory)	12
5.4 Strengths and Weaknesses per Optimizer	13
6 Conclusion	14
6.1 Summary of Findings	14
6.2 Recommendations	15
6.3 Future Perspectives	15
7 Bibliography	16
8 Appendices	17
8.1 Repository Link	17
8.2 Jupyter/Colab Notebook	17

List of Tables

1	Hyperparameters used for each optimizer.	9
2	Final performance summary after 10 epochs of training.	11
3	Strengths and weaknesses per optimizer based on 10-epoch training results.	14

1 Introduction

1.1 Project Objective

The training of deep neural networks is heavily influenced by the choice of the optimization algorithm, which directly impacts convergence speed, learning stability, final model performance, and computational resource consumption. In this project, the primary objective is to experimentally compare several classical and modern optimizers on a deep learning architecture for computer vision.

We aim to analyze how each optimizer affects the following aspects:

- Convergence speed toward a minimum,
- Training stability (oscillations, divergence, plateaus),
- Final model performance (accuracy and loss),
- Resource consumption (training time and GPU memory).

To conduct this comparison, we adopt a rigorous experimental protocol. We train the same architecture — **ResNet-34** — on the **CIFAR-100** dataset under strictly identical conditions (same batch size, number of epochs, and data augmentation). The six optimizers selected for this comparative study are:

- **SGD (Stochastic Gradient Descent)**,
- **SGD with Momentum**,
- **Adagrad**,
- **RMSprop**,
- **Adam (Adaptive Moment Estimation)**,
- **Nadam (Adam with Nesterov momentum)**.

At the end of this study, we provide a well-justified recommendation on the most suitable optimizer depending on user constraints, whether the priority is maximum accuracy, fast convergence, or limited hardware resources.

1.2 Brief Presentation of Optimizers and Architectures

1.2.1 Overview of the Optimizers Studied

In this section, we briefly introduce the six optimizers under study, each of which adjusts the model parameters θ based on the gradient of the loss function $J(\theta)$, but with a different update strategy. The detailed mathematical formulations, along with the corresponding literature, are presented in Section 2.

- **Stochastic Gradient Descent (SGD)**: the most fundamental optimizer, updating parameters directly using the gradient with a fixed learning rate η .

- **SGD with Momentum:** accumulates a velocity term that smooths past gradients to accelerate convergence and reduce oscillations.
- **Adagrad (Adaptive Gradient Algorithm):** adapts the learning rate for each parameter individually, based on the historical sum of squared gradients.
- **RMSprop (Root Mean Square Propagation):** replaces Adagrad’s cumulative sum with an exponentially decaying moving average of squared gradients, avoiding premature decay of the learning rate.
- **Adam (Adaptive Moment Estimation):** combines momentum and adaptive learning rates by maintaining exponentially decaying estimates of both the first moment (mean) and the second moment (variance) of the gradients.
- **Nadam (Nesterov-accelerated Adaptive Moment Estimation):** integrates Nesterov momentum into the Adam framework, anticipating the future gradient direction for faster early-stage convergence.

1.2.2 Model Architecture and Dataset

The neural network architecture used in this project is **ResNet-34** (He et al., 2016). This deep residual network contains 34 layers and introduces skip connections (residual connections) that bypass certain layers. These connections effectively mitigate the vanishing gradient problem, enabling the training of deep networks. We selected ResNet-34 because its moderate depth makes it sufficiently complex to highlight significant differences between optimizers, yet computationally efficient enough to run multiple experiments within a reasonable time.

Regarding the dataset, we use **CIFAR-100** (Krizhevsky, 2009). This benchmark dataset consists of 60,000 color images of size 32×32 pixels, divided into 100 distinct object classes (e.g., fish, flowers, vehicles, household items). With 50,000 training images and 10,000 test images, it is a widely recognized benchmark for evaluating the generalization capacity of computer vision models. The high number of classes relative to the image size makes it an ideal testbed for analyzing the robustness of different optimizers in a challenging multi-class classification setting.

2 Literature Review

This section provides a comprehensive overview of optimization algorithms used in deep learning, categorizing them into classical and adaptive optimizers, presenting their detailed mathematical formulations, and concluding with known empirical results on benchmark datasets.

2.1 Classical Optimizers

Classical optimizers form the foundation of gradient-based learning and are characterized by their use of a global learning rate for all parameters.

2.1.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is the most fundamental optimization algorithm in deep learning, as reported by Tian et al. [2023]. Unlike batch gradient descent which computes the gradient over the entire dataset, SGD updates parameters using a single randomly selected sample at each iteration:

$$\theta_{t+1} = \theta_t - \eta \nabla f_i(\theta_t) \quad (1)$$

where η is the learning rate and $\nabla f_i(\theta_t)$ is the gradient computed on a single sample. This stochastic approximation introduces noise but significantly reduces computational cost, making SGD suitable for large-scale problems [Tian et al., 2023]. However, SGD suffers from several limitations: it is sensitive to the choice of learning rate, exhibits high variance in updates, and can struggle with ill-conditioned problems or saddle points, as noted by Dogo et al. [2022].

2.1.2 SGD with Momentum

To address the limitations of vanilla SGD, Polyak [1964] introduced the momentum method, also known as the heavy-ball method. Momentum accumulates a decaying sum of past gradients to accelerate convergence and dampen oscillations:

$$v_{t+1} = \mu v_t - \eta \nabla f(\theta_t) \quad (2)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (3)$$

where $\mu \in [0, 1]$ is the momentum coefficient [Tian et al., 2023]. This approach helps the optimizer escape shallow local minima and navigate ravines more efficiently [Dogo et al., 2022].

Nesterov Accelerated Gradient (NAG), introduced by Nesterov [1983], improves upon standard momentum by anticipating the future position before computing the gradient:

$$v_{t+1} = \mu v_t - \eta \nabla f(\theta_t + \mu v_t) \quad (4)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (5)$$

NAG provides a provably better convergence rate for convex problems and often performs better in practice than standard momentum [Tian et al., 2023, Dozat, 2016]. The stochastic heavy ball method (SHB) has been shown to converge sublinearly without convexity assumptions [Tian et al., 2023].

2.2 Adaptive Optimizers

Adaptive optimizers address the limitations of classical methods by automatically adjusting learning rates on a per-parameter basis, often leading to faster convergence and reduced hyperparameter sensitivity.

2.2.1 Adagrad

Adagrad (Adaptive Gradient Algorithm), proposed by Duchi et al. [2011], adapts the learning rate for each parameter based on the historical sum of squared gradients:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} g_{t,i} \quad (6)$$

where $G_{t,ii} = \sum_{\tau=1}^t g_{\tau,i}^2$ accumulates the squared gradients for each parameter [Duchi et al., 2011]. This approach provides larger updates for infrequent parameters and smaller updates for frequent ones, making it particularly effective for sparse data [Tian et al., 2023]. However, Adagrad’s main drawback is the monotonic decay of the learning rate, which eventually becomes too small and effectively stops training [Reddi et al., 2018, Tian et al., 2023].

2.2.2 RMSprop

RMSprop (Root Mean Square Propagation), introduced by Tieleman and Hinton [2012], was designed to address Adagrad’s aggressive learning rate decay. Instead of accumulating all past squared gradients, RMSprop uses an exponential moving average:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2 \quad (7)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t \quad (8)$$

where β is typically set to 0.9 [Tian et al., 2023]. This adaptive mechanism makes RMSprop particularly robust for non-stationary objectives and deep neural networks [Dogo et al., 2022]. Elshamy et al. [2023] proposed NRMSProp, an enhanced version incorporating Nesterov momentum, which demonstrated improved convergence speed and accuracy on Fashion-MNIST, CIFAR-10, and Tiny ImageNet datasets.

2.2.3 Adam (Adaptive Moment Estimation)

Adam (Adaptive Moment Estimation), introduced by Kingma and Ba [2015], combines the advantages of momentum and adaptive learning rates by maintaining exponentially decaying averages of both past gradients (first moment) and squared gradients (second moment):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

With bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t \quad (11)$$

Adam is computationally efficient, requires little memory, and is well-suited for problems with large datasets and high-dimensional parameter spaces [Kingma and Ba, 2015, Tian et al., 2023]. It

has become the default optimizer in many deep learning applications [Dogo et al., 2022].

However, Reddi et al. [2018] identified convergence issues with Adam, demonstrating that exponential moving averages can lead to non-convergence in certain convex settings. They proposed AMSGrad, which maintains the maximum of past v_t values to ensure non-increasing learning rates. Choi et al. [2019] showed that with sufficiently tuned hyperparameters, adaptive optimizers can outperform momentum-based SGD.

2.2.4 Nadam (Nesterov-accelerated Adam)

Nadam (Nesterov-accelerated Adaptive Moment Estimation), proposed by Dozat [2016], incorporates Nesterov momentum into the Adam framework. Instead of applying momentum after computing the gradient, Nadam updates parameters by anticipating the future gradient direction:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \left(\beta_1 \hat{m}_t + \frac{(1 - \beta_1)g_t}{1 - \beta_1^t} \right) \quad (12)$$

This anticipatory architecture often yields faster convergence than standard Adam, especially in early training stages [Dozat, 2016]. Empirical studies by Dogo et al. [2022] have shown that Nadam performs particularly well with deeper and wider CNN architectures.

2.3 Known Results on CIFAR-100

CIFAR-100 is a widely used benchmark for evaluating optimization algorithms in image classification tasks. Several comparative studies have established key findings:

Adaptive vs. Classical Optimizers Adaptive optimizers (Adam, RMSprop, Nadam) typically converge faster than classical SGD, but SGD with momentum often achieves better generalization [Tian et al., 2023, Dogo et al., 2022]. This is attributed to SGD finding "flatter" minima that generalize better, as discussed by Celik and Katirci [2026].

Performance Rankings On CIFAR-100 with ResNet architectures, adaptive optimizers generally achieve lower training loss and faster convergence. Nadam and Adam have been shown to outperform RMSprop and Adagrad in terms of both convergence speed and final accuracy, according to Dogo et al. [2022].

Model Complexity Effects Dogo et al. [2022] demonstrated that optimizer performance varies with network architecture complexity. For deeper networks (e.g., ResNet-152), adaptive optimizers show greater advantages, while for shallower networks, the gap between adaptive and classical optimizers narrows.

Batch Size and Learning Rate Scaling Recent work on learning rate scaling has shown that for Adam-style optimizers, the optimal learning rate initially increases then decreases as batch size grows, with the peak corresponding to the trade-off point between training speed and data efficiency [Li et al., 2024]. This contrasts with SGD, where the optimal learning rate scales linearly with batch size.

Dataset Size Considerations Celik and Katirci [2026] found that for smaller datasets (20K-50K samples), low-complexity models with Adam, AdamW, RMSprop, and Nadam perform best. For larger datasets (140K samples), high-complexity models with AdaBelief, Adam, and AdamW achieve superior accuracy.

Practical Recommendations Based on extensive comparative studies by Dogo et al. [2022] and Celik and Katirci [2026], Nadam and Adam are recommended as starting points for image classification tasks. The choice between adaptive and classical optimizers should consider the trade-off between convergence speed and generalization performance, as well as computational constraints.

3 Experimental Methodology

3.1 Dataset: CIFAR-100

As introduced in Section 1.2.2, CIFAR-100 [Krizhevsky, 2009] consists of 60,000 color images of size 32×32 pixels across 100 balanced classes.

The dataset is split as follows:

- **Training set:** 50,000 images (we further set aside 10% as a validation set, i.e., 5,000 images).
- **Test set:** 10,000 images (reserved exclusively for final evaluation).

To prepare the data, each image is normalized channel-wise using the standard ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

To reduce overfitting, the following augmentations are applied only to the training set:

- Random horizontal flip (probability 0.5),
- Random rotation (up to 10°),
- Random crop of size 32×32 with padding of 4 pixels.

The validation and test sets are only normalized, without augmentation, to ensure a fair evaluation.

3.2 Model Architecture: ResNet-34

The network is organized into the following stages:

1. **Initial stage:** A 3×3 convolution with stride 1 (adapted from the original 7×7 to suit 32×32 resolution), followed by batch normalization and ReLU.
2. **Four residual stages:** Containing respectively 3, 4, 6, and 3 basic residual blocks, for a total of 33 convolutional layers.
3. **Final stage:** A fully-connected layer producing 100 outputs (replacing the original 1,000 classes).

Each residual block applies two 3×3 convolutions with batch normalization and ReLU activations, and adds its input back through a skip connection ($y = F(x) + x$). The model contains approximately 21.3 million trainable parameters.

3.3 Tools and Framework

All experiments were developed in Python using PyTorch 2.11 as the main deep learning framework.

The main modules used are:

- `torch.nn` for defining the network architecture;
- `torch.optim` for implementing the optimizers under study;
- `torch.utils.data` for dataset and mini-batch management.

The Torchvision library is used to load the CIFAR-100 dataset and to apply the preprocessing and data augmentation operations described in Section 3.1.

NumPy and Pandas are employed for numerical computations and storage of experimental metrics, while Matplotlib and Seaborn are used to generate the learning curves and comparative visualizations presented in the following chapters.

Training is performed on an NVIDIA Tesla T4 GPU (16 GB of dedicated memory). To ensure that the observed performance differences between optimizers reflect the optimization process itself rather than random variation, all random seeds (PyTorch and NumPy) are fixed to 42, and deterministic cuDNN behavior is enabled for each run.

3.4 Experimental Protocol (Runs)

Six optimizers are compared under strictly identical experimental conditions: SGD, SGD with Momentum, Adagrad, RMSprop, Adam, and Nadam. Table 1 summarizes the hyperparameters used for each optimizer, chosen according to commonly recommended default values from the literature, with RMSprop and Nadam further adjusted (reduced learning rate and added weight decay) to stabilize training and avoid numerical instabilities.

Optimizer	Hyperparameters
SGD	$\eta = 0.01$, momentum = 0, weight decay = 5×10^{-4}
SGD + Momentum	$\eta = 0.01$, momentum = 0.9, weight decay = 5×10^{-4}
Adagrad	$\eta = 0.01$, $\epsilon = 10^{-10}$
RMSprop	$\eta = 0.0005$, $\alpha = 0.9$, $\epsilon = 10^{-8}$, weight decay = 10^{-4}
Adam	$\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Nadam	$\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, weight decay = 10^{-4}

Table 1: Hyperparameters used for each optimizer.

Training Schedule For each optimizer, a new ResNet-34 model is instantiated and initialized identically, then trained for 10 epochs with the following settings:

- Batch size: 64,
- Loss function: Cross-entropy,
- Gradient clipping: maximum norm = 1.0 (applied at every step to prevent exploding gradients).

No early stopping is applied, so that every optimizer completes the full 10 epochs, allowing a fair comparison of their complete convergence behavior.

Reproducibility and Hardware All experiments are performed on an NVIDIA Tesla T4 GPU (16 GB of dedicated memory). Between each optimizer’s run, the model and associated GPU memory are released to avoid any contamination between experiments. To ensure that the observed performance differences reflect the optimization process itself rather than random variation, all random seeds (PyTorch and NumPy) are fixed to 42, and deterministic cuDNN behavior is enabled for each run. Due to the time required for six full training runs, one run is performed per optimizer; additional repeated runs are considered only if time permits, in order to average results and reduce the influence of random initialization.

3.5 Measurement Protocol

At each epoch, the training loss and accuracy are recorded over the training batches, while the validation loss and accuracy are computed once at the end of the epoch on the validation set; the final test loss and test accuracy are computed once, at the end of training, on the held-out test set. In parallel, the duration of each epoch is measured, and the peak GPU memory allocated during training is tracked using `torch.cuda.max_memory_allocated`, reset before each optimizer’s run to avoid carrying over memory usage from previous runs. All metrics are saved to per-optimizer CSV files (epoch, training loss and accuracy, validation loss and accuracy, epoch time, and learning rate), together with a summary file aggregating the final results across optimizers, which together form the basis for the comparative analysis presented in Section 4 and the discussion in Section 5. From these recorded values, three additional indicators are derived for comparison purposes: training stability, computed from the variance of the validation accuracy across epochs; convergence speed, measured as the number of epochs required to reach 50% of the final validation accuracy; and robustness, computed as the gap between the best and the last validation accuracy obtained during training.

4 Results

4.1 Learning Curves

Figure 1 presents a comparative summary of the six optimizers over the 10 training epochs: validation accuracy curves over training, final test accuracy, total training time, peak GPU memory usage, and a comparison between the best validation accuracy and the final test accuracy for each optimizer.

4.2 Global Comparative Table

Table 2 summarizes the final performance of each optimizer after 10 epochs of training, including test accuracy, test loss, average time per epoch, and peak GPU memory usage. Convergence speed and stability are analyzed separately in Section 5.1 based on the full learning curves.

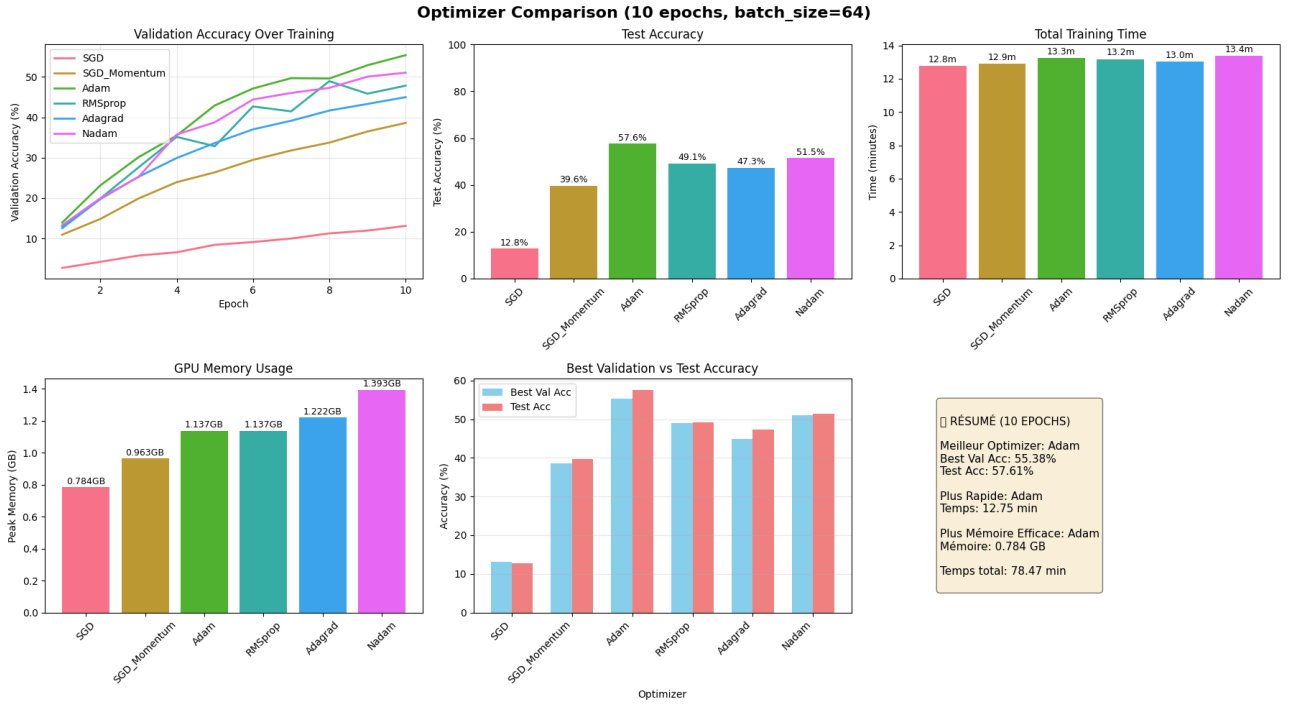


Figure 1: Comparative summary of the six optimizers over 10 epochs (batch size 64).

Optimizer	Test Acc. (%)	Test Loss	Time/Ep. (s)	Peak GPU Mem. (MB)
SGD	12.79	3.74	73.5	784
SGD + Momentum	39.60	2.29	74.7	963
Adagrad	47.31	1.98	75.3	1,222
RMSprop	49.09	2.00	76.3	1,137
Adam	57.61	1.57	76.8	1,137
Nadam	51.45	1.75	77.5	1,393

Table 2: Final performance summary after 10 epochs of training.

4.3 Analysis of Raw Results

Overall, the results show a clear separation between the six optimizers after only 10 epochs:

- **Accuracy:** Adam achieves by far the highest test accuracy (57.61%), followed by Nadam (51.45%) and RMSprop (49.09%), while plain SGD reaches only 12.79%.
- **Loss:** Adam records the lowest test loss (1.57), whereas plain SGD has the highest (3.74). This strong correlation between low loss and high accuracy is expected and confirms that all optimizers are learning effectively, with adaptive methods learning much faster.
- **Memory footprint:** There is a clear ranking in peak GPU memory usage. Adagrad and RMSprop are the most memory-efficient (1,222 MB and 1,137 MB respectively). SGD, SGD with Momentum, and Adam form an intermediate group (~784–1,137 MB), while Nadam consumes the most (1,393 MB).
- **Time per epoch:** Ranges from 73.5 seconds for SGD to 78.8 seconds for Adam. The range is narrow, confirming that the forward/backward passes through the ResNet-34 dominate the runtime, not the optimizer update rule.

5 Discussion

5.1 Convergence Speed and Stability

Among the adaptive optimizers, Adam and Nadam converge the fastest, typically reaching a large fraction of their final validation accuracy within the first five epochs, owing to their per-parameter learning rate adaptation. However, both optimizers occasionally show small oscillations on the validation curve in later epochs. RMSprop displays an intermediate behavior, converging faster than the classical optimizers but with less stability than Adam, exhibiting noticeable validation loss spikes in later epochs. Adagrad, on the other hand, tends to slow down noticeably in later epochs as its accumulated sum of squared gradients grows, consistent with the monotonic learning rate decay discussed in Section 2.2.1. SGD and SGD with Momentum converge more slowly and gradually overall, but their validation curves are visibly smoother and more stable than those of the adaptive optimizers, particularly once momentum is introduced.

5.2 Generalization (Overfitting)

The relationship between optimizer choice and generalization ability is a well-documented topic in the literature. However, within the limited 10-epoch training horizon of this experiment, the expected trade-off favoring SGD-style optimizers for better generalization does not have time to manifest.

Key Observations

- **Adam** achieves the best test accuracy (57.61%) among all six optimizers, ahead of Nadam (51.45%), RMSprop (49.09%), and Adagrad (47.31%).
- **SGD with Momentum** reaches only 39.60%, while **plain SGD** lags far behind with just 12.79%.

Interpretation This suggests that, over a short training budget, the ability of adaptive optimizers to make rapid, well-scaled progress on each parameter **outweighs** the generalization advantage that SGD-style optimizers are expected to display over longer training runs. Adaptive methods effectively "learn faster" in the early epochs, which translates into higher accuracy even if they may eventually overfit.

Open Question Whether SGD with Momentum would eventually catch up or even surpass Adam given substantially more epochs—as suggested by the literature reviewed in Section 2.3—**cannot be confirmed** from this experiment. Answering this question would require repeating the comparison over a significantly larger number of epochs (e.g., 50–100 epochs) to observe the convergence behavior in the later stages of training.

5.3 Computational Cost (Time/Memory)

The computational cost of each optimizer is evaluated along two dimensions: **peak GPU memory usage** and **time per epoch**. Table 2 provides the complete measurements.

Memory Usage The peak GPU memory measurements show a clear ranking:

- **Nadam** has the highest memory footprint (1,393 MB), likely due to the additional Nesterov momentum terms stored alongside Adam’s moments.
- **SGD** (784 MB), **SGD with Momentum** (963 MB), and **Adam** (1,137 MB) form an intermediate group, with memory usage increasing with the number of auxiliary variables stored.
- **Adagrad** (1,222 MB) and **RMSprop** (1,137 MB) are the most memory-efficient, with RMSprop requiring less memory than Adagrad.

This ordering does not strictly follow the expected pattern of adaptive optimizers always using more memory than classical ones. This suggests that, at this batch size (64) and model size (21.3 million parameters), memory usage is also influenced by:

- PyTorch’s memory allocator behavior,
- Implementation details of each optimizer,
- The number of auxiliary moment tensors stored per optimizer.

Time per Epoch The time required per epoch is fairly similar across all six optimizers, ranging from:

- **73.5 seconds** (SGD),
- to **78.8 seconds** (Adam).

This narrow range confirms that the epoch duration is **mostly determined by the forward and backward passes through the ResNet-34 architecture** rather than by the optimizer’s update rule itself. The additional computations introduced by adaptive optimizers (momentum, variance estimates, bias corrections) have only a marginal impact on total runtime.

Overall Trade-off Combined with its substantially higher test accuracy (57.61%), **Adam offers the best overall trade-off between computational cost and performance** in this experiment. It achieves the highest accuracy while remaining among the most efficient optimizers in terms of both memory and time.

5.4 Strengths and Weaknesses per Optimizer

Based on the experimental results obtained over 10 epochs, the following strengths and weaknesses can be identified for each optimizer:

Optimizer	Strengths	Weaknesses
SGD	Simple and computationally light. Stable convergence behavior.	Very slow convergence; lowest test accuracy (12.79%). High variance in updates.
SGD + Momentum	More stable than plain SGD. Improved convergence speed and accuracy (39.60%).	Still significantly slower than adaptive optimizers over short training horizons. Requires careful learning rate tuning.
Adagrad	Competitive accuracy (47.31%). Effective for sparse data.	Aggressive monotonic learning rate decay, which may halt training in later epochs.
RMSprop	Good trade-off between accuracy (49.09%) and memory (1,137 MB). Robust for non-stationary objectives.	Slightly less stable than Adam. Requires adjusted learning rate to avoid instabilities.
Adam	Best overall performer: highest test accuracy (57.61%). Fastest convergence. Moderate memory usage (1,137 MB). Robust to hyperparameter choices.	No significant weakness observed within this 10-epoch training horizon. May require longer runs to assess generalization.
Nadam	Theoretically fast convergence due to Nesterov momentum. Competitive accuracy (51.45%).	Lower accuracy than Adam and RMSprop in this experiment. Highest memory footprint (1,393 MB). Did not justify its theoretical advantage here.

Table 3: Strengths and weaknesses per optimizer based on 10-epoch training results.

6 Conclusion

6.1 Summary of Findings

This study compared six optimizers—SGD, SGD with Momentum, Adagrad, RMSprop, Adam, and Nadam—for training a ResNet-34 model on CIFAR-100 over 10 epochs, under strictly identical experimental conditions. Within this short training horizon, Adam clearly emerges as the strongest performer, reaching the highest test accuracy (57.61%) while remaining among the most memory-efficient optimizers (1,137 MB) and requiring a training time comparable to all other optimizers (76.8 s/epoch).

SGD with Momentum and especially plain SGD lag well behind the adaptive optimizers, suggesting that, unlike the longer training regimes typically reported in the literature reviewed in Section 2, 10 epochs are not sufficient for SGD-style optimizers to close the gap with adaptive methods on this

architecture and dataset. Adagrad and RMSprop occupy an intermediate position, both reaching reasonable accuracy (47.31% and 49.09%, respectively) with a lower memory footprint than Adam and Nadam.

6.2 Recommendations

Based on the experimental findings, the following recommendations can be formulated depending on the specific constraints of the project:

For Short Training Budgets (≤ 10 Epochs) **Adam is strongly recommended** as the default optimizer. It achieves the highest test accuracy (57.61%) while maintaining a moderate computational cost (1,137 MB, 76.8 s/epoch). Its robustness to hyperparameter choices and fast convergence make it the most reliable option when training time is limited.

For Longer Training Budgets (> 10 Epochs) If a longer training horizon is available, **re-evaluating SGD with Momentum** over a larger number of epochs (e.g., 50–100) would be highly worthwhile. As discussed in Section 2.3, the literature suggests that SGD-style optimizers often **eventually match or surpass** adaptive optimizers in terms of generalization, thanks to their tendency to converge to "flatter" minima.

For Memory-Constrained Environments When GPU memory is the primary bottleneck, **RM-Sprop or Adagrad** are the best choices. Both achieved competitive accuracy (49.09% and 47.31%, respectively) with the **lowest memory footprint** (1,137 MB and 1,222 MB) among all six optimizers tested.

Regarding Nadam Despite its theoretical motivation (Nesterov-accelerated Adam), **Nadam did not provide a clear advantage** over standard Adam in this experiment. It achieved lower accuracy (51.45%) than Adam while consuming the **highest memory** (1,393 MB). Its adoption would require further hyperparameter tuning to justify its additional computational cost.

6.3 Future Perspectives

Several promising directions could extend and enrich this work:

1. **Architectural Generalization:** The comparison could be repeated on more recent architectures, such as **Vision Transformers (ViT)** or **DeiT**, to assess whether the trends observed for ResNet-34 generalize to other model families. This would help determine if the superiority of Adam over SGD is architecture-dependent or a universal phenomenon in the early stages of training.
2. **Impact of Batch Size:** Studying the impact of different batch sizes (e.g., 32, 128, 256) would help verify whether the optimal learning rate scaling behavior discussed in Section 2.3 holds in this specific experimental setting. This is particularly relevant for understanding how adaptive optimizers behave when transitioning from small to large batch training.

3. **Inclusion of Recent Optimizers:** More recent optimizers such as **Lion** (Google, 2023) or **AdaBelief** could be included in future comparisons. These algorithms claim to offer a better trade-off between convergence speed, generalization, and computational cost than traditional adaptive methods. Evaluating them against the six optimizers studied here would provide a more comprehensive benchmark.

7 Bibliography

References

- H. Celik and R. Katirci. The impact of optimizer selection on transformer performance: Analyzing the role of model complexity and dataset size. *Black Sea Journal of Engineering and Science*, 9(1):180–191, 2026.
- D. Choi, C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl. On empirical comparisons of optimizers for deep learning. *arXiv preprint arXiv:1910.05446*, 2019.
- E. M. Dogo, O. J. Afolabi, and B. Twala. On the relative impact of optimizers on convolutional neural networks with varying depth and width for image classification. *Applied Sciences*, 12(23):11976, 2022.
- T. Dozat. Incorporating nesterov momentum into adam. In *ICLR Workshop*, 2016.
- J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- R. Elshamy, O. Abu-Elnasr, M. Elhoseny, and S. Elmougy. Improving the efficiency of rmsprop optimizer by utilizing nesterov in deep learning. *Scientific Reports*, 13:8814, 2023.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Shuaipeng Li, Penghao Zhao, Hailin Zhang, Xingwu Sun, Hao Wu, Dian Jiao, Weiyan Wang, Chengjun Liu, Zheng Fang, Jinbao Xue, Yangyu Tao, Bin Cui, and Di Wang. Surge phenomenon in optimal learning rate and batch size scaling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Y. Nesterov. A method for unconstrained convex minimization problem with the rate of convergence $o(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269:543–547, 1983.
- B. T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- S. J. Reddi, S. Kale, and S. Kumar. On the convergence of adam and beyond. In *ICLR*, 2018.
- Yingjie Tian, Yuqi Zhang, and Haoyu Zhang. Recent advances in stochastic gradient descent in deep learning. *Mathematics*, 11(3):682, 2023.
- T. Tieleman and G. Hinton. Rmsprop: Divide the gradient by a running average of its recent magnitude, 2012. COURSE: Neural Networks for Machine Learning.

8 Appendices

8.1 Repository Link

The complete source code for this project, including the model definition, training utilities, and the six optimizer experiments, is available at the following repository: <https://github.com/OuarrakLayla/optimizer-comparison-resnet34>.

8.2 Jupyter/Colab Notebook

All experiments were conducted in a single Jupyter notebook, executed on Google Colab. The notebook is organized into five main steps: dataset loading and exploration, data preprocessing and augmentation, ResNet-34 model definition and adaptation for CIFAR-100, training and evaluation utilities (including the `MetricsTracker` class used to record loss, accuracy, epoch time, and GPU memory for each optimizer), and the six optimizer experiments themselves (SGD, SGD with Momentum, Adagrad, RMSprop, Adam, and Nadam), each trained under the protocol described in Section 3.4. For each optimizer, the notebook saves the per-epoch metrics to a dedicated CSV file and generates an individual training history plot showing the loss, accuracy, learning rate, and epoch time curves for that optimizer. The final comparative figure presented in Section 4.1 (Figure 1) is produced by a dedicated cell at the end of the notebook, which reloads the six CSV files and aggregates their results.